Gatopía — Documentación y evidencias de seguridad

1. Descripción del sistema

Gatopía es un prototipo de sistema IoT para monitorear vibraciones mediante un sensor SW-420 conectado a un Arduino UNO R4 WiFi. El dispositivo se comunica inalámbricamente con un servidor desarrollado en Python, que utiliza SQLite para almacenar información del sistema y eventos de vibración.

Una aplicación Android permite consultar el estado de la alarma, revisar el historial y, según los permisos de la cuenta, activar o desactivar el sistema.

2. Objetivo de seguridad

Proteger el acceso a las funciones administrativas, restringir el envío de eventos desde dispositivos no autorizados y conservar un registro de los acontecimientos detectados.

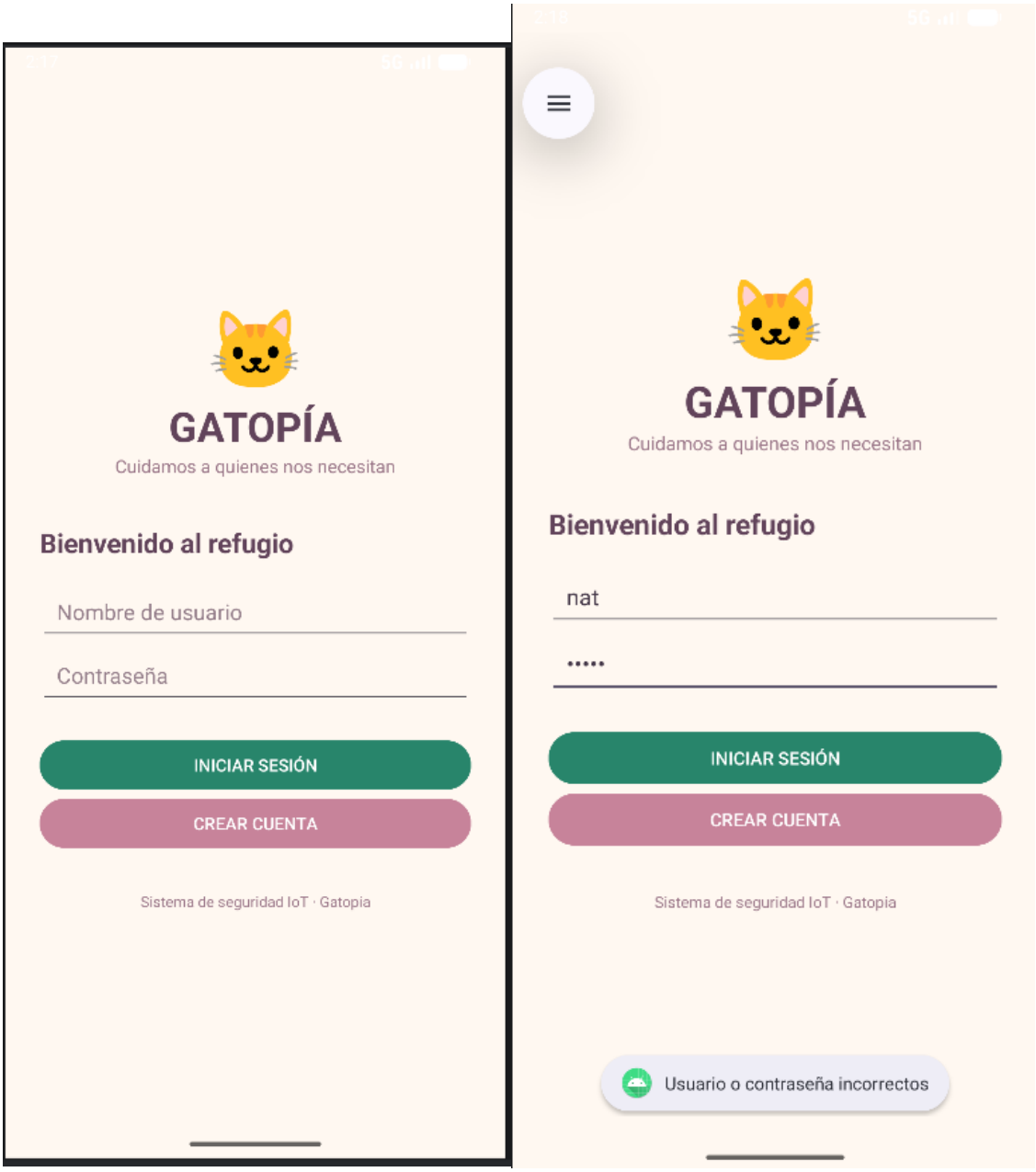
3. Medidas de seguridad implementadas

3.1 Autenticación de usuarios

El servidor dispone de un mecanismo de inicio de sesión mediante nombre de usuario y contraseña.

La autenticación se realiza en el servidor, evitando depender exclusivamente de comprobaciones realizadas en la aplicación Android.

Evidencia:



3.2 Protección de contraseñas

El servidor utiliza PBKDF2 con sal para derivar valores de verificación de contraseñas, evitando almacenarlas directamente en texto plano.

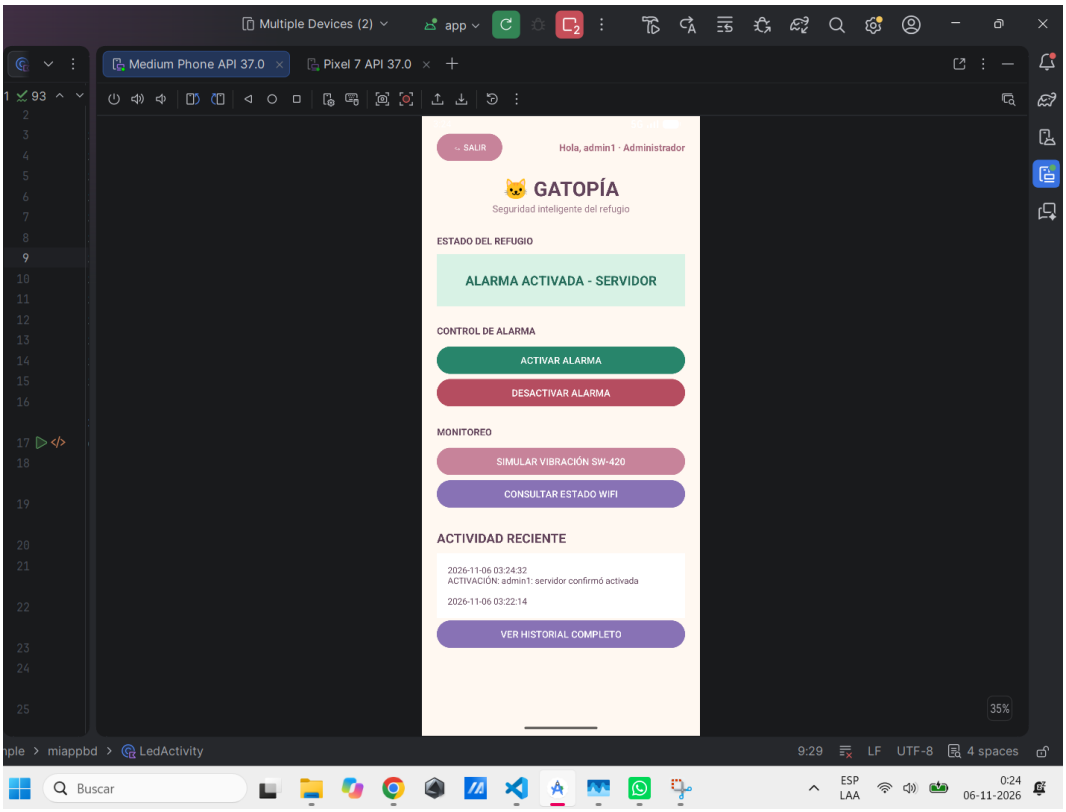
Evidencia sugerida: fragmento del código Python donde se genera y verifica el hash de contraseña, ocultando cualquier dato sensible.

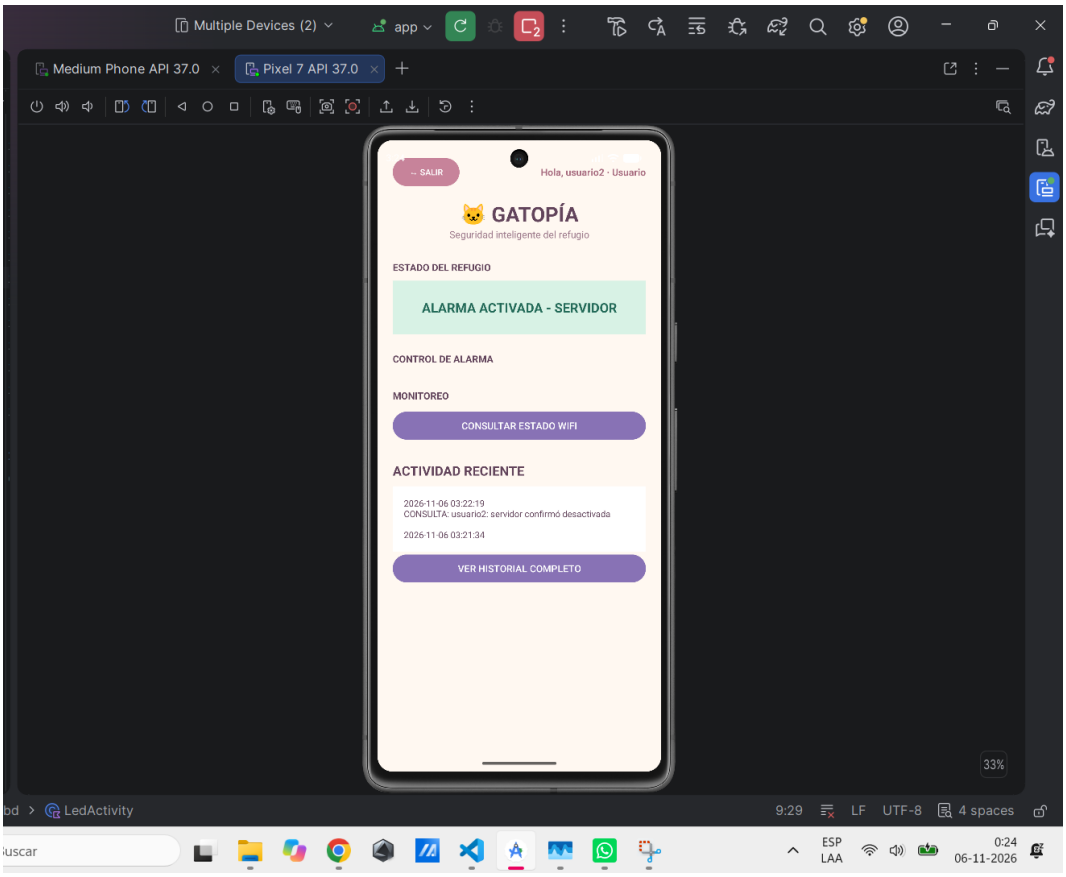
3.3 Control de acceso por roles

El sistema diferencia entre cuentas administrativas y cuentas de usuario.

Las funciones de activación y desactivación de la alarma requieren autorización administrativa, que debe verificarse en el servidor.

Evidencia:


Multiple Devices (2)
app
Medium Phone API 37.0
Pixel 7 API 37.0
SALIR
Hola, admin1 · Administrador
GATOPÍA
Seguridad inteligente del refugio
ESTADO DEL REFUGIO
ALARMA ACTIVADA - SERVIDOR
CONTROL DE ALARMA
ACTIVAR ALARMA
DESACTIVAR ALARMA
MONITOREO
SIMULAR VIBRACIÓN SW-420
CONSULTAR ESTADO WIFI
ACTIVIDAD RECIENTE
2026-11-06 03:24:32
ACTIVACIÓN: admin1: servidor confirmó activada
2026-11-06 03:22:14
VER HISTORIAL COMPLETO
35%
LedActivity
9:29
LF
UTF-8
4 spaces
Buscar
ESP
LAA
0:24
06-11-2026


Multiple Devices (2)
app
Medium Phone API 37.0
Pixel 7 API 37.0
SALIR
Hola, usuario2 · Usuario
GATOPÍA
Seguridad inteligente del refugio
ESTADO DEL REFUGIO
ALARMA ACTIVADA - SERVIDOR
CONTROL DE ALARMA
MONITOREO
CONSULTAR ESTADO WIFI
ACTIVIDAD RECIENTE
2026-11-06 03:22:19
CONSULTA: usuario2: servidor confirmó desactivada
2026-11-06 03:21:34
VER HISTORIAL COMPLETO
33%
LedActivity
9:29
LF
UTF-8
4 spaces
ESP
LAA
0:24
06-11-2026

3.4 Autenticación del dispositivo IoT

El Arduino envía una clave de dispositivo mediante la cabecera HTTP X-Gatopia-Key al registrar una vibración.

El servidor comprueba esa clave antes de aceptar la solicitud.

3.5 Registro de eventos

Las vibraciones detectadas se registran en el servidor y pueden consultarse desde la aplicación Android.

Esto permite mantener un historial de actividad y comprobar que las alertas se reciben.

Evidencia:



3.6 Protección de la red local

El prototipo utiliza una red WiFi local para comunicar el Arduino, el servidor y los dispositivos Android.

Durante las pruebas se configuraron reglas del firewall de Windows para controlar el acceso al puerto utilizado por el servidor.

4. Limitaciones y riesgos identificados

El sistema es un prototipo académico y presenta limitaciones que deben declararse:

- **Comunicación HTTP sin cifrado:** las solicitudes no utilizan TLS, por lo que no existe protección criptográfica de los datos durante el transporte.
- **Clave estática del Arduino:** si se expone, podría utilizarse para intentar suplantar el dispositivo.
- **Dependencia de la red local:** la comunicación requiere conectividad entre el Arduino y el servidor.
- **Disponibilidad del servidor:** si Python deja de ejecutarse, no se pueden registrar nuevas alertas de manera centralizada.
- **Sesiones en memoria:** las sesiones activas pueden perderse cuando se reinicia el servidor.
- **Protección de consultas:** es necesario revisar el acceso a los endpoints de lectura de estado e historial para evitar divulgar información innecesariamente.

5. Pruebas de seguridad propuestas

Prueba	Resultado que debe verificarse
Iniciar sesión con contraseña incorrecta	El servidor rechaza el acceso
Ingresar con cuenta de usuario	No puede activar ni desactivar la alarma
Enviar una vibración sin clave de dispositivo válida	El servidor rechaza la solicitud
Registrar una vibración autorizada	El evento se almacena y aparece en Android
Interrumpir la conexión WiFi	El sistema informa el fallo de comunicación
Reiniciar el servidor	Se verifica la recuperación del servicio y el comportamiento de las sesiones

Las pruebas que aún no se hayan ejecutado deben marcarse como **pendientes**, no como aprobadas.

6. Justificación del uso de SQLite

Se seleccionó SQLite por tratarse de un sistema de monitoreo de alcance académico, con un servidor central y una cantidad reducida de usuarios y dispositivos.

SQLite ofrece persistencia de datos sin necesidad de instalar y administrar un servidor de base de datos independiente. Además, permite almacenar usuarios, estado de la alarma y eventos mediante tablas y consultas SQL.

Su integración con Python facilita el desarrollo del prototipo, mientras que su uso en Android permite mantener información local cuando la aplicación lo requiere.

La arquitectura centraliza los registros compartidos en la base de datos SQLite del servidor. Esto es importante porque los dispositivos Android deben consultar una fuente común para mostrar el mismo estado y los mismos eventos.

Para una implementación de mayor escala sería conveniente evaluar una base de datos con capacidades más amplias de concurrencia, administración y disponibilidad.

7. Conclusión

Gatopía incorpora controles básicos de autenticación, autorización y validación del dispositivo IoT, junto con un registro centralizado de eventos.

Las pruebas funcionales realizadas demuestran la comunicación entre Arduino, Python y Android. Sin embargo, la seguridad completa del sistema requiere pruebas adicionales, especialmente sobre acceso no autorizado y protección de la comunicación.

Como mejoras futuras se proponen HTTPS/TLS, gestión segura de secretos, fortalecimiento de las sesiones y restricciones adicionales sobre los endpoints de consulta.